

Many-to-many Object Relationships Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/p3-oo-many-to-many-lab>  <https://github.com/learn-co-curriculum/p3-oo-many-to-many-lab/issues/new>

Learning Goals

- Create a Many-to-many relationship using an intermediary class.
- Write aggregate methods.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

In this lab we will implement a one-to-many relationship between a `Author` , `Book` , and `Contract` .

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.

Instructions

Create a Book class that has the following attributes: `title` (string)

Create an Author class that has the following attributes: `name` (string)

Create a Contract class that has the following properties: `author` (Author object), `book` (Book object), `date` (string), and `royalties` (int).

All classes should also keep track of `all` members using a class variable.

- The author property should be an instance of the Author class, while the book property should be an instance of the Book class. The date property should be a string that represents the date when the contract was signed, while the royalties property should be a number that represents the percentage of royalties that the author will receive for the book.
 - All setters should `raise Exception` upon failure.

The `Author` class should have the following methods:

- `contracts(self)` : This method should return a list of related contracts.
- `books(self)` : This method should return a list of related books using the `Contract` class as an intermediary.
- `sign_contract(book, date, royalties)` : This method should create and return a new `Contract` object between the author and the specified book with the specified date and royalties
- `total_royalties()` : This method should return the total amount of royalties that the author has earned from all of their contracts.

The `Contract` class should have the following methods:

- A class method `contracts_by_date(cls, date)`: This method should return all contracts that have the same date as the date passed into the method.

Resources

- [Python classes](https://docs.python.org/3/tutorial/classes.html)  (<https://docs.python.org/3/tutorial/classes.html>)

This tool needs to be loaded in a new browser window

Load Many-to-many Object Relationships Lab (CodeGrade) in a new window